Multi-Leader & Leaderless Replication

conflicts, CRDTs and quorums

Watch the 4-min video

In one line: multi-leader lets several regions accept writes, at the price of write conflicts; leaderless (Dynamo-style) sends each write to N replicas and needs W + R > N to read the latest value, with read repair and hinted handoff cleaning up the replicas that missed it.

1 The problem: one leader, two continents

A collaborative notes app has users in Europe and the US. With one leader in Virginia, every keystroke from Berlin crosses the Atlantic before it is saved: roughly a **90 ms** round trip. And if Virginia goes down, nobody can write at all. What if both regions could accept writes?

2 Multi-leader replication

Each region has its own leader. It accepts **local writes** and replicates them **asynchronously** to the other leaders.

- **Fast:** writes are local, a few ms instead of a transatlantic round trip.
- **Resilient:** a whole region can fail while the other keeps working.
- **The price:** two leaders can change the same data at the same time.

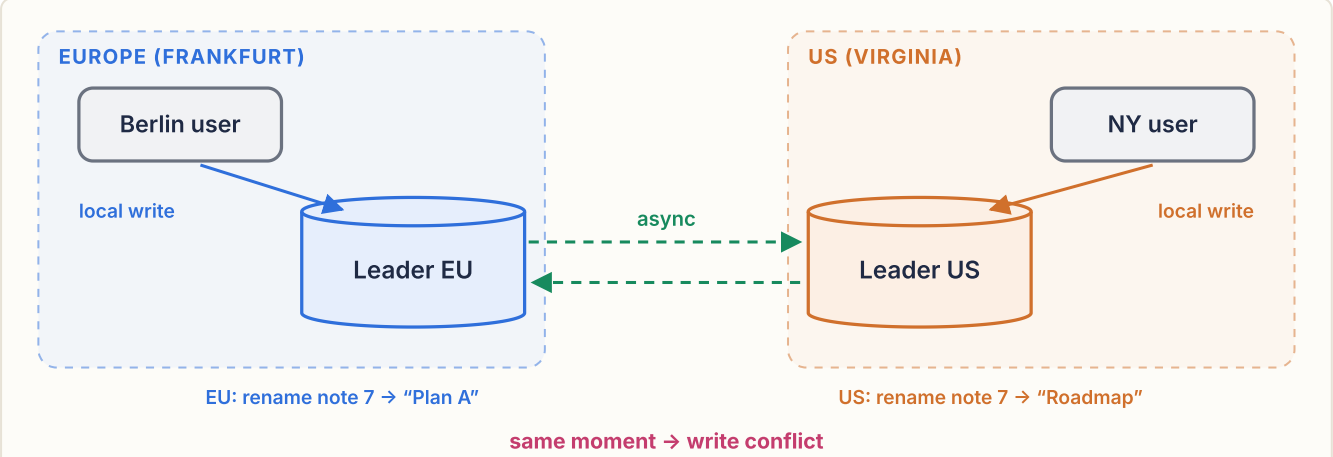


Figure 1. Both regions accept the rename before either hears about the other. Neither write is wrong; together they conflict.

3 Resolving write conflicts

Two users rename the same note, one in each region. Options:

- **Last write wins (LWW):** keep the version with the latest timestamp and silently throw the other away. Clock skew makes it worse: a server whose clock runs 50 ms fast wins every close race.
- **Merge** both values, or **ask** the application or the user to choose.
- **CRDTs** (conflict-free replicated data types): counters, sets and text sequences designed to merge automatically and always converge to the same result on every replica. Libraries like **Yjs** and **Automerge** power collaborative editors this way.



Figure 2. Three ways to resolve a conflict. Only last-write-wins loses data, and it does so silently.

WORKED EXAMPLE: A CRDT COUNTER

A like-counter is replicated in EU and US. Instead of one number, each region keeps **its own count**: EU = 3, US = 5, and the value is the sum, 8. EU gets 2 likes (EU = 5), US gets 1 (US = 6), at the same moment. Each side merges by taking the max of each region's count: both end at EU = 5, US = 6, total **11**. With last write wins on a single number, one side would write 10, the other 9, and one write would be lost.

4 Leaderless replication and quorums

Dynamo-style databases such as Cassandra (and Riak, ScyllaDB) remove the leader entirely. The client (or a coordinator node) sends each write to all **N** replicas, and the write succeeds when **W** of them confirm. Reads ask **R** replicas. If **W + R > N**, the set read always overlaps at least one replica that has the latest write.

```
W + R > N  ·  common setting: N = 3, W = 2, R = 2
```

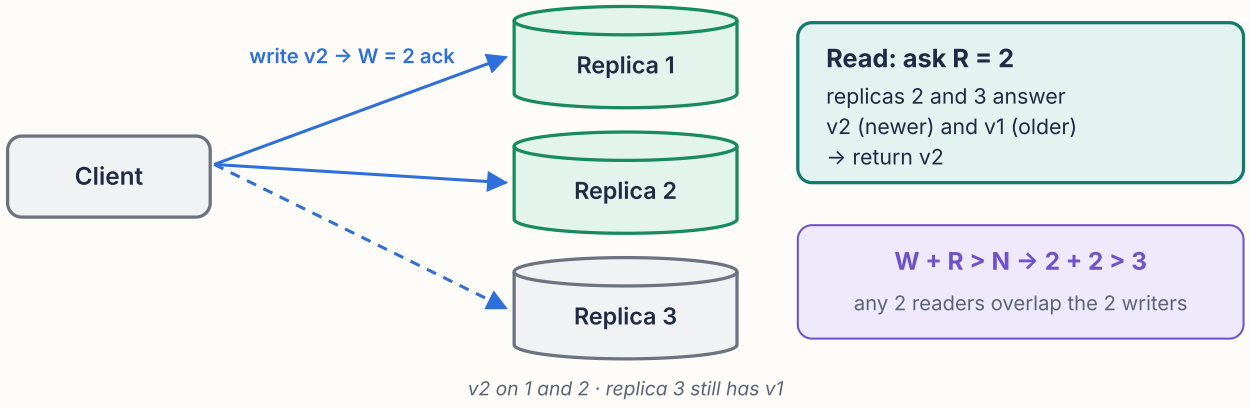


Figure 3. N = 3, W = 2, R = 2. Whichever two replicas you read, at least one has v2, and the newer version wins.

Setting (N = 3)	Effect
W = 2, R = 2	balanced; reads and writes both survive 1 replica down
W = 3, R = 1	fast reads, but any single replica down blocks writes
W = 1, R = 3	fast writes, but any single replica down blocks reads
W = 1, R = 1	fastest, but 1 + 1 is not > 3: reads can be stale

5 Repairing replicas that missed writes

Some replicas will miss writes, for example while they are down. Three mechanisms fix this:

- **Read repair.** A read collects versions from several replicas, notices one is stale, returns the newest value, and writes it back to the stale replica.
- **Anti-entropy.** A background process compares replicas using **Merkle trees** (hash trees, so only differing ranges are transferred) and catches data that is rarely read.
- **Hinted handoff.** If a target replica is unreachable, another node accepts the write with a **hint** saying who it is really for. When the replica comes back, the hint is handed off. This is a **sloppy quorum**: it keeps writes available, but it is not a strict overlap guarantee.

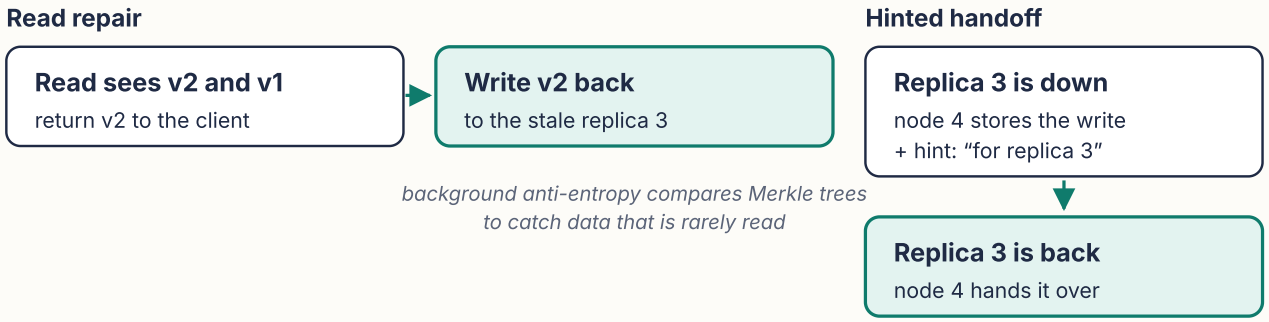


Figure 4. Read repair fixes data on the read path; hinted handoff keeps writes flowing while a replica is away.

6 The risks, and how teams tame them

Multi-leader has real risks: conflicts on every shared record, silent data loss with last write wins, broken **unique constraints** and **auto-increment IDs** (two regions can both create user “priya” or ID 1001), and behaviour that is hard to reason about.

A common mitigation: give each document a **home region**, so all its writes go to one leader, and use **CRDTs** only where real concurrent editing is required.

COMMON MISTAKES

- Choosing multi-leader for latency without a plan for conflicts.
- Trusting last write wins for data that matters (it silently drops updates).
- Picking W and R with W + R ≤ N and still expecting fresh reads.
- Treating a sloppy quorum as a strict guarantee.

7 Key terms

Multi-leader: several nodes accept writes, replicate async

CRDT: data type that merges automatically

Read repair: fix stale replicas during a read

Write conflict: concurrent changes to the same data

Leaderless: client writes to N replicas directly

Anti-entropy: background sync via Merkle trees

Last write wins: keep latest timestamp, drop the rest

Quorum (W, R): acks needed to write / replies to read

Hinted handoff: another node holds the write for later

8 Interview questions

> **Q1. What are the risks of multi-leader replication?** BASIC
> Concurrent writes to the same data in different regions conflict, and resolving them with last write wins silently loses updates. Unique constraints and auto-increment IDs can break, and behaviour is hard to reason about.

Q2. How does read repair work? BASIC

A read queries several replicas, detects stale versions, returns the newest value to the client, and writes that newest value back to the stale replicas.

Q3. Why does W + R > N give fresh reads? BASIC

Any W replicas that took the write and any R replicas you read must share at least one node when W + R > N. With N = 3, W = 2, R = 2, two writers and two readers always overlap.

Q4. N = 3, W = 2, R = 2. One replica is down. Can you still read and write? SCENARIO

Yes: two replicas are up, enough for W = 2 and R = 2. With two down, both fail (unless sloppy quorums with hinted handoff accept writes on other nodes).

Q5. Design writes for a notes app used in EU and US, with occasional co-editing. SCENARIO

Give each note a home region and send its writes to that leader (simple, no conflicts for most notes). For live co-editing, use a CRDT text type (Yjs or Automerge) so concurrent edits merge instead of overwriting.

Q6. Two regions both increment a like-counter with last write wins. What goes wrong, and the fix? SCENARIO

Both read 9, both write 10: one like is lost. Use a CRDT counter (one count per region, value = sum, merge = max per region), or route increments to one home region.

Q7. Does W + R > N guarantee you always read the latest value? TRICKY

Not strictly. Sloppy quorums, concurrent writes, a write that succeeded on fewer than W nodes then failed, and clock-based conflict resolution can all return older data. It is a strong practical guarantee, not linearizability.

Q8. Why not use wall-clock timestamps to order writes across regions? TRICKY

Clocks drift: if one server runs 50 ms fast, its writes win every race within 50 ms, even when they were actually earlier. Use version vectors to detect concurrency, or CRDTs to merge.

Next: SD-E-022 · Database Sharding Fundamentals backendroadmap.com